Exp.No:6 Error Correction at Data Link Layer.

Aim:

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature.

Create sender program with below features

1. Input to sender file should be a text of any length. Program should convert the text to binary
2. Apply hamming code concept on the binary data and add redundant bits to it.
3. Save the output in a file called channel.

Create a receiver program with below features

1. Receiver program should read the input from channel file
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of error.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

Code:

```
def calc_redundant_bits(m):
    for r in range(m+1):
        if(2**r) >= (m+r+1):
            return r

def main():
    data = input("Enter data min: ").strip()
    m = len(data)
    r = calc_redundant_bits(m)
    n = m+r
    print(f"Number of r needed : {r}")
    print(f"Total frame length: {n}")
    code = ['0'] * (n+1)
    j = 0
    for i in range(1,n+1):
        if(i & (i-1)) != 0:
            code[i] = data[(j+1)]
            j += 1
    for i in range(r):
        parity_pos = 2**i
        val = 0
        for pos in range(1,n+1):
            if (pos & parity_pos)!=0 and pos!=
                                    parity_pos:
                if code[pos] == '1':
                    val ^= 1
        code[parity_pos] = str(val)
transmitted_str = ''.join(code[1:])[::-1]
print(f"Data to be transmitted : {transmitted_str}")
change = input("In Transmitted bit change").strip().upper
if change == "YES":
    received_str = input(f"Enter Corrupted data")
    if len(received_str) = n:
```

```
        print(f"Error: received data is exactly {m} with
                                    bits long!")
        return
    received_code = [int('0')] + list(received_str[::-1])
else:
    received_code = list(code)

if error_position == 0:
    print("\n verification result : No error detected")
else
    print("\n verification result : Error detected
    print(f"wrong bit at position: {error position}")
    received_code[error_position] = '0' if received_code[
        error_position] == '1' else '1'
    corrected_str = "".join(received_code[1:][::-1])
    print(f"corrected data stream : {corrected_str}")
if __name__ == "__main__":
    main()
```

Output:

① Enter data bits : 1011
Number of Redundant bits (r) needed : 4
Total frame length (m+r) : 10
Data to be transmitted : 1011 0001
Transmitted bit change ? (Yes/No) : YES
Enter the corrupted data stream : 1011 0101
verification result : Error detected
wrong bit is at position : 9
corrected data streams : 1011 0001

② Enter data bits : 1011
Number of redundant bits (r) needed : 3
total data frame length (m+r) : 7
Data to be transmitted : 1010101
Transmitted bit change ? (Yes / No) : No
verification result : No error

Result:

Thus the hamming code was implemented successfully.